

AI Blog Nest – Generate High Quality blog content using AI

Govt Arts College For Women , Krishnagiri

Team ID : SWTID-2026-1691

Team Size : 5

Team Leader : BHUVANESHWARI. K

Team member : AARTHI. V

Team member : ABIRAMI. R

Team member : ANUSHREE. P

Team member : ASWIN. R

Introduction

Blog Nest is a full-stack blogging web application that allows users to create accounts, write and publish blog posts, and engage with other users' content through comments and likes. The platform is designed to be simple, fast, and easy to use for both casual writers and regular readers.

The core goals of the project are to:

- Provide a clean, distraction-free space for users to write and publish articles.
- Allow readers to discover content through categories, tags, and search.
- Support user engagement through comments, likes, and follows.
- Give administrators tools to moderate content and manage the platform.

Blog Nest is built using a modern web stack — a React front end, a Node.js/Express back end, and a MongoDB database — following a typical three-tier architecture that separates presentation, business logic, and data storage.

Note: this report assumes a React / Node.js / MongoDB stack. If your project uses a different stack, this section and the diagrams below can be updated accordingly.

Software Requirements

Operating System: Windows 10/11, macOS, or Linux (supports cross-platform operations).

Node.js (v16 or above): Runtime ecosystem running server logic and routing infrastructure.

npm (v8 or above): Node package manager to control operational dependencies.

Express.js: Lightweight routing web framework to construct backend RESTful entry points.

MongoDB: Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.

Postman: API verification toolkit to assert schema validation responses across administrative route shields.

Code Editor: Visual Studio Code or similar IDE environment.

Hardware Requirements

Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.

RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).

Storage: 1 GB of available disk workspace.

Project architecture

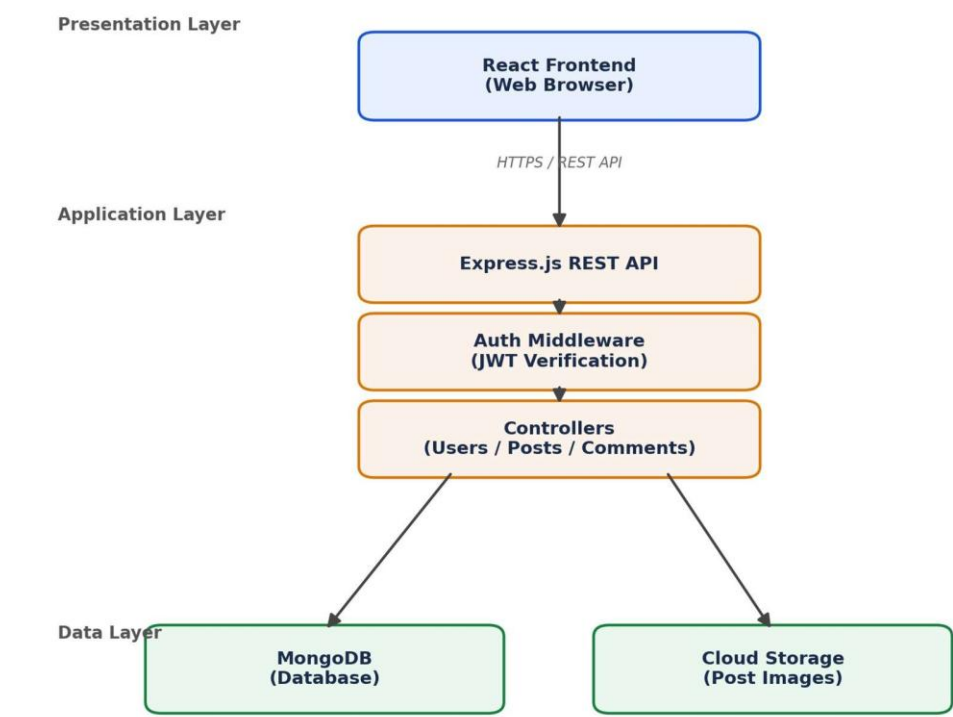


Figure 1: Blog Nest system architecture

Key Components

Component	Responsibility
React Frontend	Renders pages, manages UI state, calls backend APIs
Express REST API	Routes incoming requests to the right controller
Auth Middleware	Validates JWT tokens, protects private routes
Controllers	Contain business logic for users, posts, comments, likes
MongoDB	Persistent storage for all application data

ER Diagram

The data model centers on Users who write Posts, which can receive Comments and Likes, and are organized under Categories.

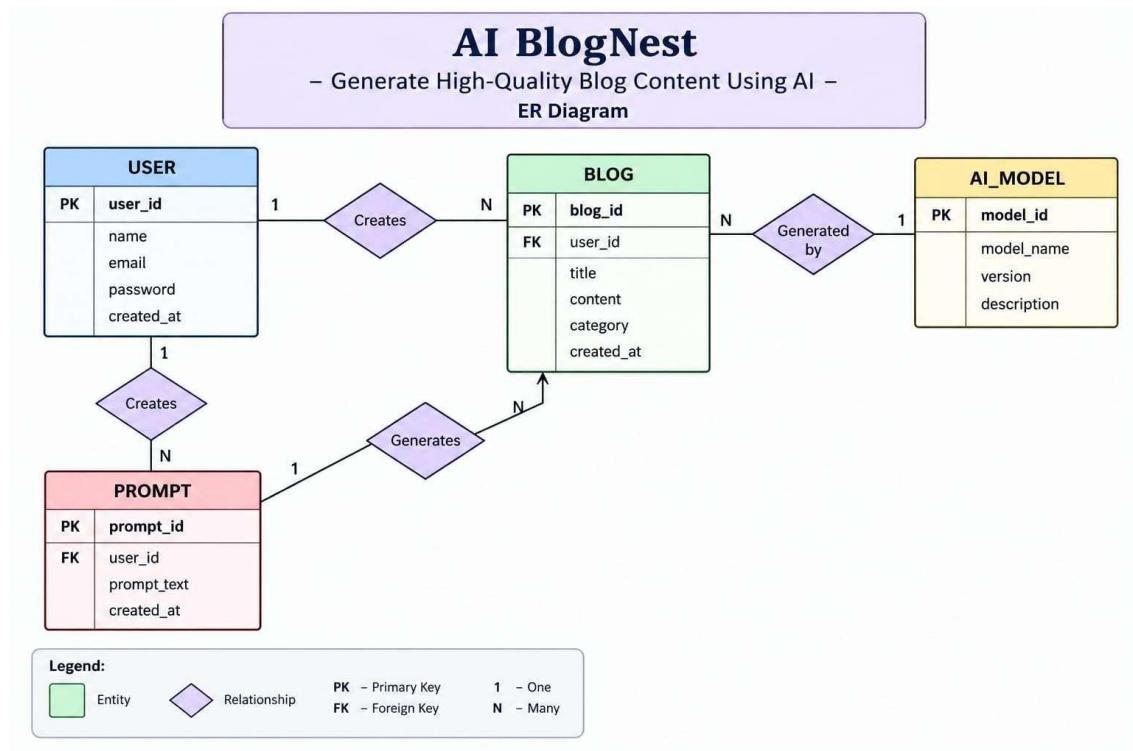


Figure 2: Entity-Relationship diagram

User Entity

- `_id` (Primary Key - ObjectId)
- `name` (String, Required)
- `email` (String, Required, Unique)
- `password` (String, Hashed, Required)
- `role` (String, Default: "reader")

Post Entity

- `_id` (Primary Key - ObjectId)
- `userID` (Foreign Key referencing User Collection)
- `name` (String, Required - represents Title)
- `photo` (Object/String pathway link)
- `message` (String - Core content body)
- `likes` (Number, Default: 0)
- `timestamps` (True)

Relationship Summary

- A User can write many Posts, Comments, and Likes (1-to-many).

- A Post belongs to one User (author) and one Category, but can have many Comments and Likes.
- A Category can classify many Posts.

Development Team Roles

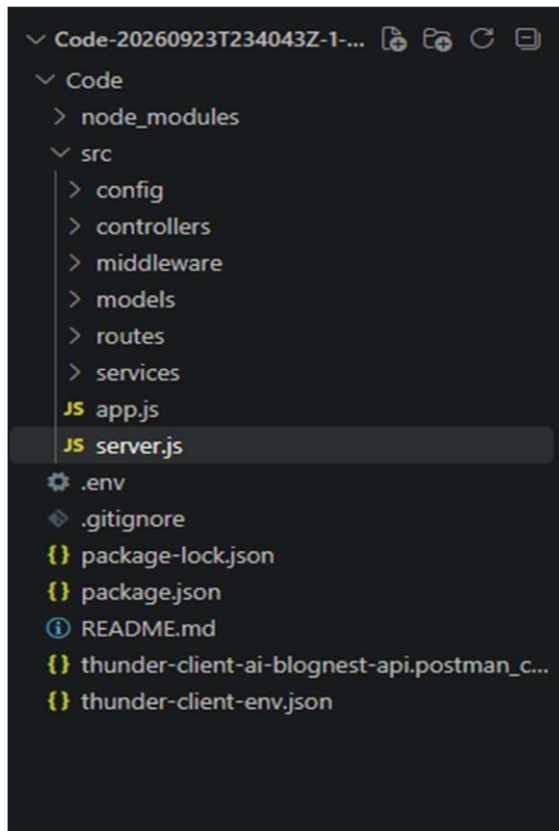
Role	Responsibilities
Frontend developer	Builds UI components, handles state management, integrates with backend APIs, ensures responsive design
Backend Developer	Designs REST APIs, implements authentication, writes business logic, manages database schema
Database Administrator	Designs the schema/ER model, optimizes queries, manages backups and data integrity.
UI/UX Designer	Designs wireframes and mockups, ensures a consistent and usable interface

Key Features

- Role-Based Route Guarding: Strict token processing blocks unauthorized tiers from modifying core settings.
- CRUD Blog Execution Engine: Complete schema management over content definitions, status states, and asset assignment paths.
- Categorization System: Native tag indices facilitate filtering and querying for enhanced discoverability.
- Comment Moderation Logic: Protected entry points for updating moderation flags (approved, pending, spam).
- Sanitization Safeguards: Payload parameter sanitation prevents XSS vectors and NoSQL parameter manipulation injections.
- I-Driven Blog Generation Pipeline: Integrates external Large Language Model (LLM) abstractions and context-aware prompt templates to synthesize full-length, structured blog posts from high-level thematic inputs.
- Intelligent Content Summarization Engine: Leverages semantic compression layers to parse extensive article payloads, dynamically generating concise abstracts and automated metadata summaries.

Project setup and configuration

Creating project folder



Server setup(npm init)

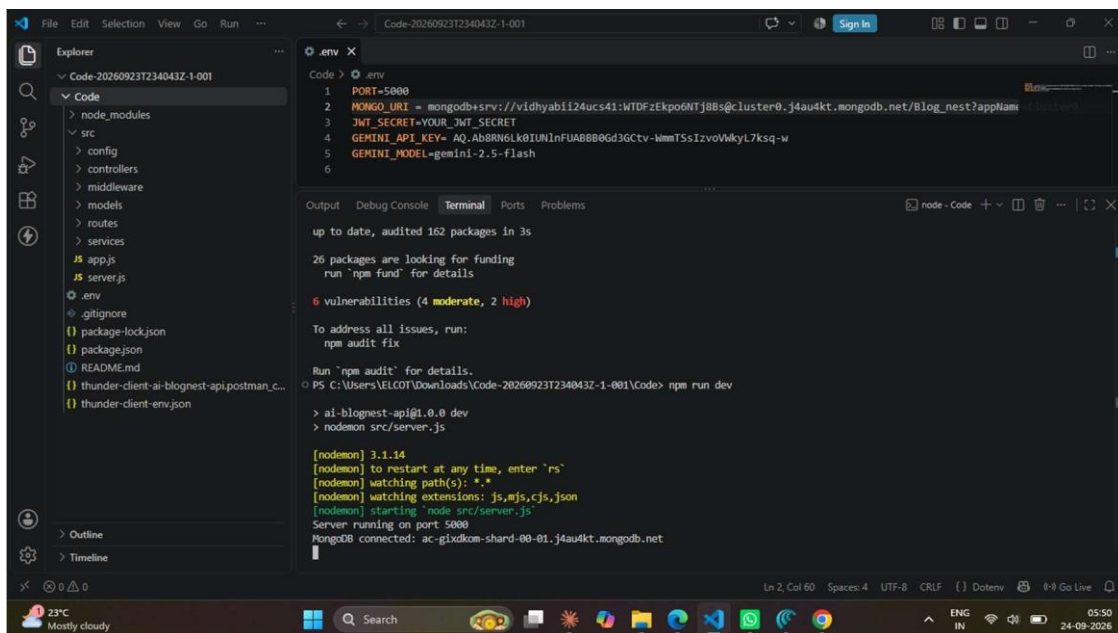
```
JS server.js X
Code > src > JS server.js > ...
1  const dotenv = require('dotenv');
2  const dns = require('dns');
3  const app = require('./app');
4  const connectDB = require('./config/db');
5
6  dotenv.config();
7
8  dns.setServers(['8.8.8.8', '8.8.4.4']);
9
10 connectDB();
11
12 const PORT = process.env.PORT || 5000;
13 app.listen(PORT, () => {
14   console.log(`Server running on port ${PORT}`);
15 });
16
```

App.js

```
1 const express = require('express');
2 const cors = require('cors');
3 const morgan = require('morgan');
4 const authRoutes = require('./routes/authRoutes');
5 const blogRoutes = require('./routes/blogRoutes');
6 const aiRoutes = require('./routes/aiRoutes');
7 const errorMiddleware = require('./middleware/errorMiddleware');
8
9 const app = express();
10
11 app.use(cors());
12 app.use(express.json());
13 app.use(morgan('dev'));
14
15 app.use('/api/auth', authRoutes);
16 app.use('/api/blogs', blogRoutes);
17 app.use('/api/ai', aiRoutes);
18
19 app.use(errorMiddleware);
20
21 app.get('/', (req, res) => {
22   res.json({ message: 'AI BlogNest API is running' });
23 });
24
25 module.exports = app;
```

API TESTING

Mongodb connection:



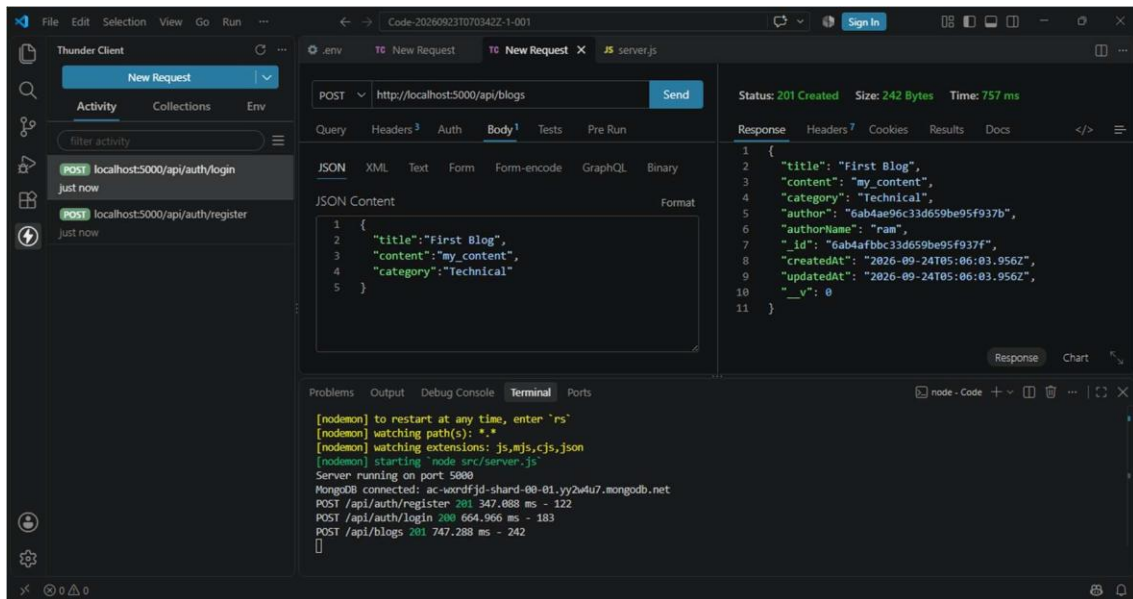
The screenshot shows a VS Code editor with a file explorer on the left and a terminal on the right. The file explorer shows a project structure with folders like 'node_modules', 'src', 'config', 'controllers', 'middleware', 'models', 'routes', 'services', and files like 'app.js', 'server.js', '.env', 'gitignore', 'package-lock.json', 'package.json', 'README.md', 'thunder-client-ai-blognest-api.postman_collection.json', and 'thunder-client-env.json'. The terminal shows the output of running 'npm run dev' in a Node.js environment. It displays the MongoDB connection string and the server running on port 5000.

```
Code > .env
1 PORT=5000
2 MONGO_URI = mongodb+srv://vidhyab124ucs41:WDFzEkpo6NTj88s@cluster0.j4au4kt.mongodb.net/Blog_nest?appName=cluster0
3 JWT_SECRET=YOUR_JWT_SECRET
4 GEMINI_API_KEY= AQ.AB8RN6lk0IUNlnFUABBB0Gd3Gctv-1mmTSsIzvoVmkyl7ksq-w
5 GEMINI_MODEL=gemini-2.5-flash
6

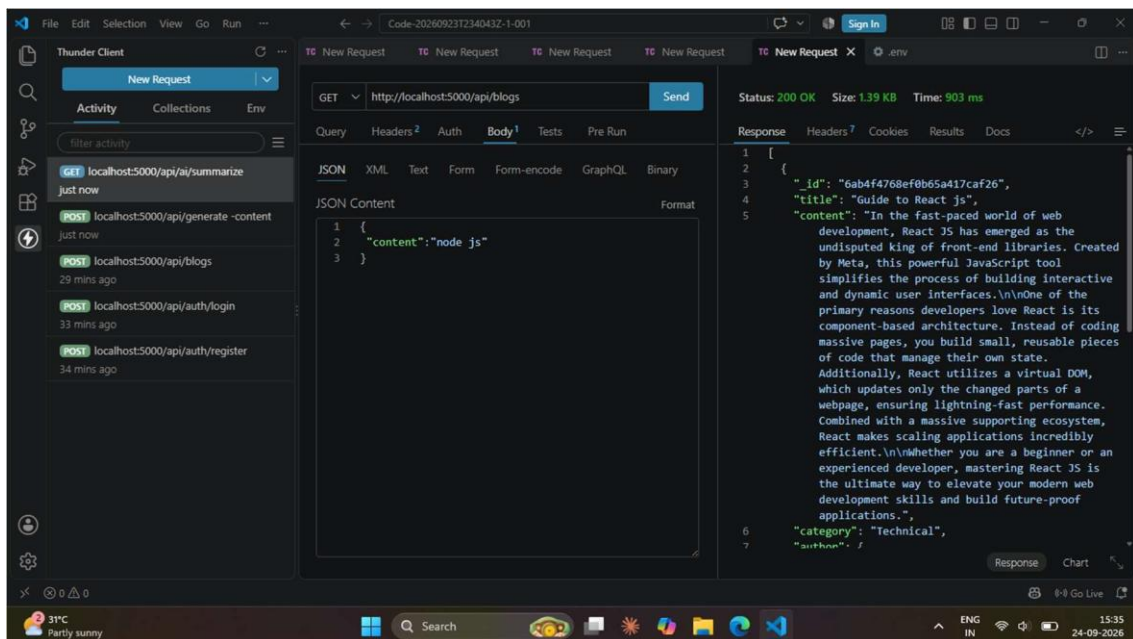
Output Debug Console Terminal Ports Problems
up to date, audited 162 packages in 3s
26 packages are looking for funding
run 'npm fund' for details
6 vulnerabilities (4 moderate, 2 high)
To address all issues, run:
npm audit fix
Run 'npm audit' for details.
PS C:\Users\ELCOT\Downloads\Code-20260923T234043Z-1-001\Code> npm run dev
> ai-blognest-api@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter 'rs'
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting 'node src/server.js'
Server running on port 5000
MongoDB connected: ac-gixdcom-shard-00-01-j4au4kt.mongodb.net
```

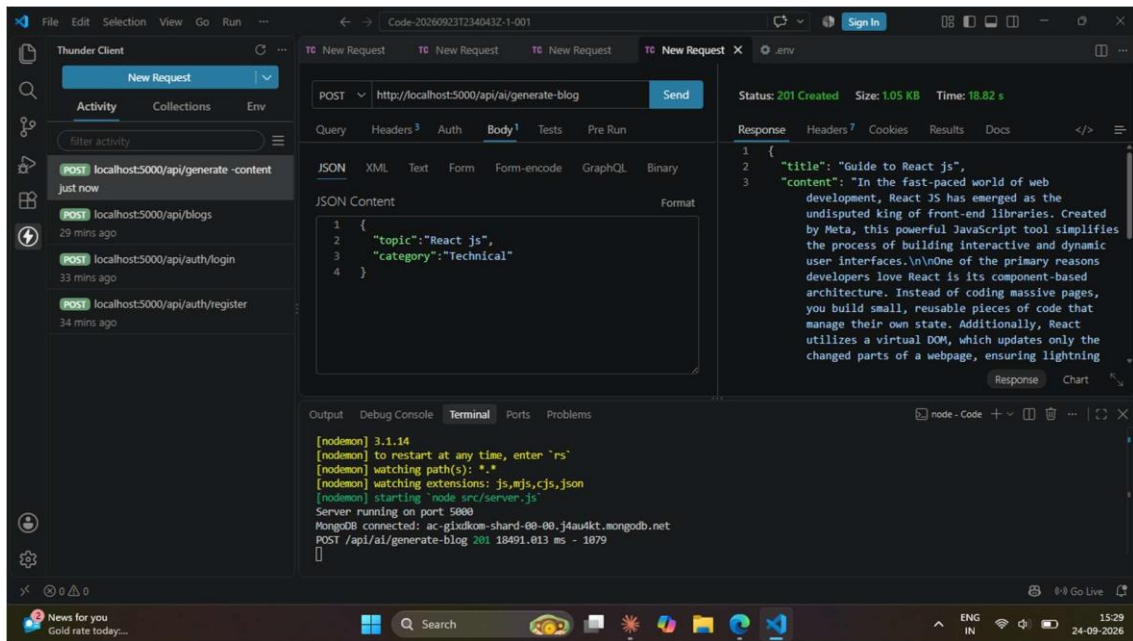
User registration:



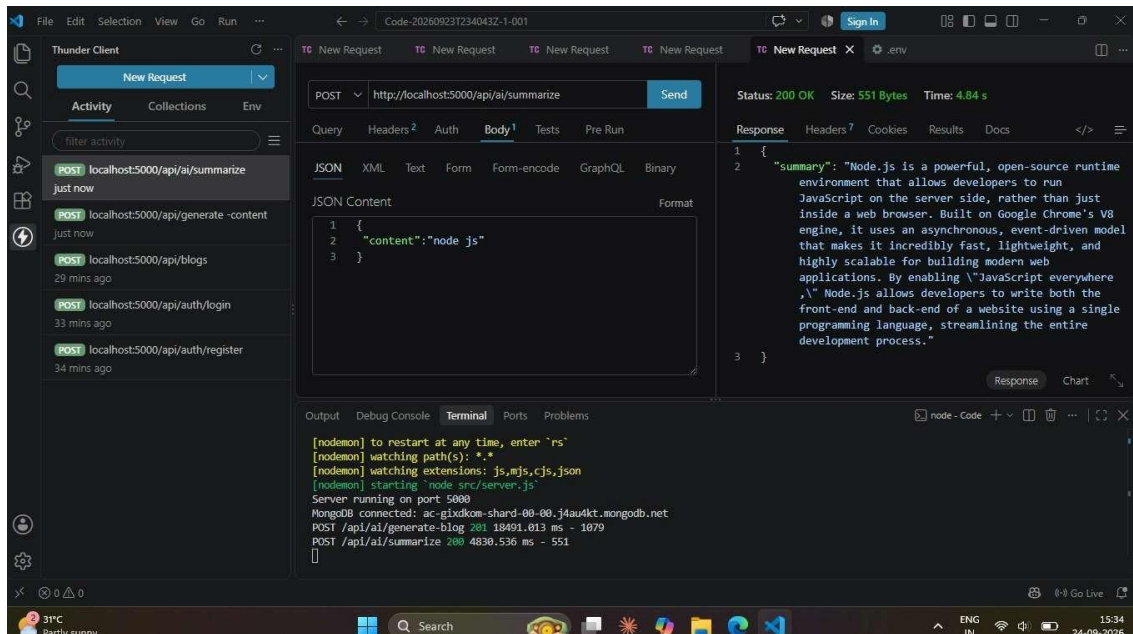
Fetch Blogs:



Generate AI Blogs:



Summarize Blogs:



Blog_nest Demo

Demo video:

video.mp4



API testing

API Testing